

How to do it

Implementation Guidelines for multi-language support feature for applications with Ya3dag_LanguageExtract

26.08.2026 RR First edition of this manual

Supporting switchable multi-language functionality for applications can be a major challenge. Having implemented multi-language support for two applications, I am making this work available here for others.

Table of Contents

1. Supported programming languages and operating systems.....	1
2. Requirements.....	1
3. Overview: How It Works.....	2
4. Creating language definitions.....	2
5. Collect language definitions.....	3
6. Make translations.....	5
7. Using language definitions.....	6
8. Description of the package's directories.....	8
9. Notes on the compiler used.....	8

1. Supported programming languages and operating systems

The command-line tool for extracting language definitions are written in C++. The same applies to the code for using language definitions.

The command-line tool is intended for use in a Windows batch file. The code for using language definitions must be adapted to the applications. My applications run on Windows. Linux applications can certainly use the code as well.

2. Requirements

It's important that you have a basic understanding of character sets ([ASCII](#), [ANSI code page](#), [UTF-8](#)). If possible, you should enable UTF-8 support in the compiler for your applications.

3. Overview: How It Works

1. Creating language definitions
Language definitions are created in the code at the places where strings are used. Specially formatted text strings are used for this purpose.
These language definitions take the following form: "&tag=string".
Each tag must be unique. The tag is used in the code for language translation to retrieve the translation.
2. Collect language definitions
All language definitions are collected as text in one or more files using the command-line tool 'Ya3dag_LanguageExtract'. These files are stored in a separate subdirectory (e.g., named '**Language-Scan**').
3. Create Translations
Translations are generated by creating a copy of the '**Language-Scan**' directory and the files it contains. The directory is named after a language, e.g., 'German'.
A separate subdirectory is created for each supported language.
Of course, the language definitions in this directory still need to be translated.
4. Using language definitions
When the application starts, data is loaded from one of the language subdirectories. To speed up searches, the language definitions are sorted by tag value.
Whenever a translation is needed, the **LangStringLookup()** function is called with a language definition as an argument.
Example: `sprintf( Temp, LangStringLookup( "&GUI_Main_Info_CuVal3=Position:" ));`

4. Creating language definitions

Language definitions are created in the code at the places where strings are used. Specially formatted text strings are used for this purpose.

These language definitions take the following form: "&tag=string".

- * Must always be enclosed in double quotation marks
- * The first character must be a '&'
- * After that the tag follows. Use alphanumeric characters, _ or -. NO blanks.
- * After '=' the string follows.

Longer language definitions can be spread over multiple lines:

```
"&tag-long="
"First line\n"
"Next line"
"Last line"
```

A best practice is to define the language definitions and their translations right where they are used in the code.

Note: **LangStringLookup()** is the function that performs the translation in the application.
Here are a few examples:

```
// Replace a function's string argument with a call to LangStringLookup() and a language definition.
```

```
pGUI_DiffUses = new YaIPS_Fl_Box( x, y, xx, yy, LangStringLookup( "&GUI_Main_InfBox1=Info" ));
```

```
pRadioButTemp->tooltip( LangStringLookup( "&GUI_Main_InfMeas9h=Measures horizontal distance" ));
```

```
sprintf( errbuffer, LangStringLookup( "&RGB_Color_NoRGB=No color image!" ));
```

```
// Language definitions can also be used as constant strings in tables. The only important thing is
// that they are translated using the LangStringLookup() function when used.
```

```
char * YaIPS_ColMod_LUT_table[] =
{ (char *)&Utils_ColMod_LUT_None=---",
  (char *)&Utils_ColMod_LUT_01=Color steps",
  (char *)&Utils_ColMod_LUT_02=Color gradient",
  ~~~~~~
  (char *)&Utils_ColMod_LUT_10=Iron",
  (char *)&Utils_ColMod_LUT_11=Cold Hot",
  (char *)&Utils_ColMod_LUT_12=Hue",
};

add( LangStringLookup( YaIPS_ColMod_LUT_table[ i]);
```

```
// If identical strings are used frequently, they can also be collected centrally as macros in an include file.
```

```
#define LANGDEF_SETTINGS      LangStringLookup( "&Text_Settings=Settings")

pGUI_Parameter->tooltip( LANGDEF_SETTINGS_POINTS);
```

5. Collect language definitions

All language definitions are collected as text in one or more files using the command-line tool ‘**Ya3dag_LanguageExtract**’. These files are stored in a separate subdirectory (e.g., named ‘**Language-Scan**’).

The command-line tool ‘**Ya3dag_LanguageExtract**’ uses these arguments:

```
Ya3dag_LanguageExtract [-i string] [-f] [-n] output_file [files_to_add]
    -i ignore if string is part of filename to add
    -f output language tags after each file
    -F output language tags after each file to a separate output file
    -n no sorting of the language tags
    -o print language tag optimization hints to console
    -t print test prints to console
```

Ya3dag_LanguageExtract recursively scans all subdirectories in ‘files_to_add’. Strings after the characters ‘//’ are skipped. For files with the extension ‘.txt’ or ‘.cfg’ also strings after the character ‘;’ are skipped.

Ya3dag_LanguageExtract is used as a command-line tool in a Windows batch file. A best practice is to copy the **LanguageExtract** tool to the directory where the Windows batch file is located. Another best practice is to create subdirectories for the different languages within this directory.

Example 1:

```
@echo off
rem File: ScanLanguage-Initial.bat
rem
@echo on

rem make subdirecory for language scan
mkdir Language-Scan

rem Usecase 1: Scan output into one output file
Ya3dag_LanguageExtract -f -n Language-Scan\OneOutputFile.txt Test-Src\*.txt Test-Src\*.h Test-Src\*.cpp

rem Usecase 1: An output file for each scanned file
Ya3dag_LanguageExtract -F -n Language-Scan\Translation.txt Test-Src\*.txt Test-Src\*.h Test-Src\*.cpp
```

```
rem User can inspect output of the tools
pause
```

Example 2:

```
rem Recursic scans all .h and .cpp files in the 'src' subdirectory.
Ya3dag_LanguageExtract -F -n Language-Scan\Translation.txt ..\src\*.h ..\src\*.cpp
```

Example 3:

```
rem All text files can be scanned
Ya3dag_LanguageExtract Language-Scan\Translation-GameIntro.txt ..\pics\Intro\GameIntro.cfg
Ya3dag_LanguageExtract -f -n Language-Scan\Translation-Scripts.txt ..\gamedata\AScr_*.txt
```

Below is a language file generated by **Ya3dag_LanguageExtract**. The language definitions are collected here for later use.

```
;-----
; Test-Source.cpp
;-----
;
; 3 entries
;
Tag-printf                                "Hello world."

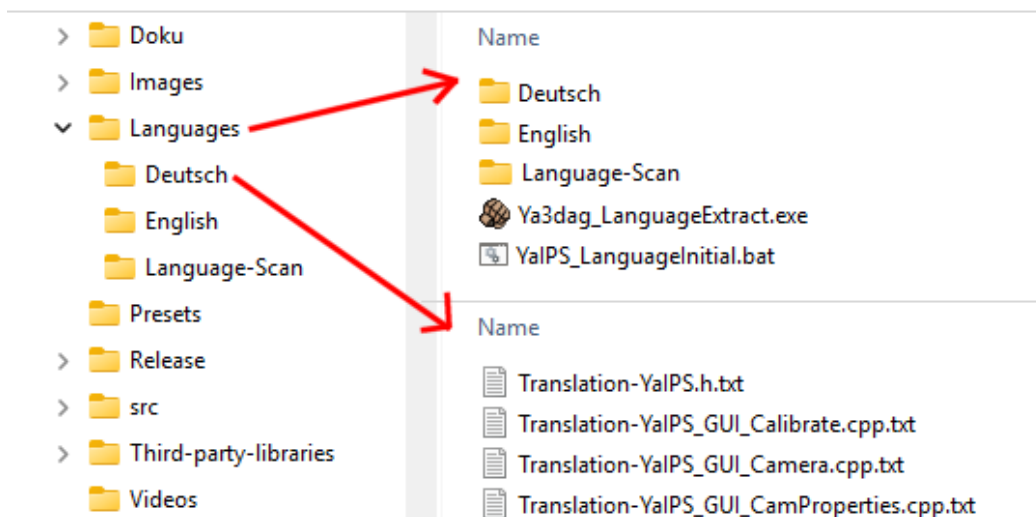
Tag-String                                "Any text for a string"

Help-Text-123                             "Bla bla bla line 1\n"
                                           "Line 2\n"
                                           "Last line"

;
;
; End Of File
```

Below is a screenshot of the language directory from one of my applications.

Everything is organized in one place here: the Windows batch file used to create the initial language files and the translations.



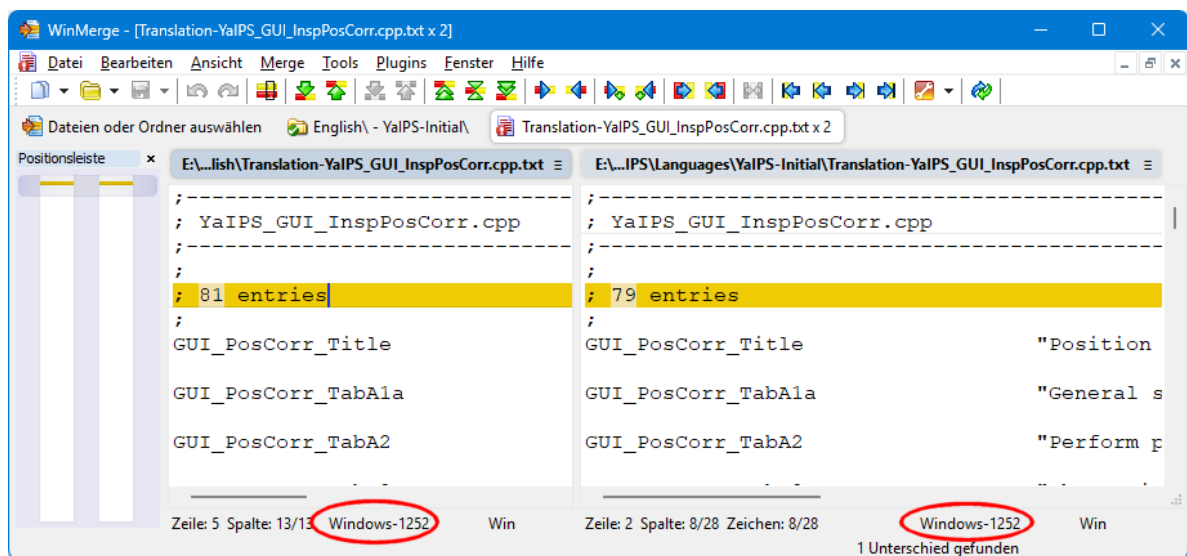
6. Make translations

Translations are generated by creating a copy of the '**Language-Scan**' directory and the files it contains. The directory is named after a language, e.g., 'German'. A separate subdirectory is created for each supported language. Of course, the text in the language files still needs to be translated.

If the language definition texts in the code were all written in the same language - for example, English - it is sufficient to simply rename the copied directory. In this example, rename it to 'English.'

My proven approach for updating language definitions - for example, when releasing a new version of your application:

- After you've finished making changes to your application, create an update of the collected language definitions in the '**Language-Scan**' directory.
 - Use the [WinMerge](#) application to identify the changes in the '**Language-Scan**' language files and the native language of the language definitions. 'English', for example.
 - Using a second **WinMerge** application, you can now compare the '**Language-Scan**' with another language directory to be used as a reference. 'German', for example.
- Based on the changes displayed in the first **WinMerge** application, you can synchronize the changes and translations in the second **WinMerge** application.



Important: The character set or code page that WinMerge uses for the file must be compatible with your applications.

7. Using language definitions

When the application starts, data is loaded from one of the language subdirectories. To speed up searches, the language definitions are sorted by tag value. This allows the **LangStringLookup()** function to quickly access the loaded language data.

Part of this repository is a source file named “Language-Translate.cpp” for reading language files and translating language definitions. You can use this file in your application.

At the beginning of this file, you'll find instructions on what adaptations you need to make. The most important adaptation is retrieving a list of files in a directory.

Below is an excerpt from the corresponding header file:

```
/*=====

Lang_FreeData()

Free up all the memory allocated for the most
recently loaded language data.

Return:

    None

=====
*/

void Lang_FreeData();

/*=====

LangLoadTranslations()

Load the language data for a specific language

Arguments:

    char *pLanguage          Name of the directory containing the language data

    int NewLineCharForMultipleStringLines
                             If set, insert new line characters at end of multiple string
lines.

                             Example: tag_xyz  "Line 1"
                             "Line 2"
                             "Line 3"

Return:

    0      OK
    != 0   Error

=====
*/

int LangLoadTranslations( char *pLanguage, int NewLineCharForMultipleStringLines =
false);

/*=====

Lang_Init()

Load the language files for the last selected language.
Use this after starting your application.
```

Arguments:

```
int NewLineCharForMultipleStringLines
    If set, insert new line characters at end of multiple string
lines.
```

```
Example: tag_xyz "Line 1"
"Line 2"
"Line 3"
```

NOTE: The last language used is stored in a global variable

Return:

None

```
=====
*/
```

```
void Lang_Init( int NewLineCharForMultipleStringLines = false);
```

```
/*=====
```

```
LangStringLookup()
```

Look up a language definition.

Arguments:

```
char *pString    Look up this language definition.
                  It should be in the format "&tag=string".
```

Return:

```
NULL            This only happens if the 'pString' argument is 'NULL'.
```

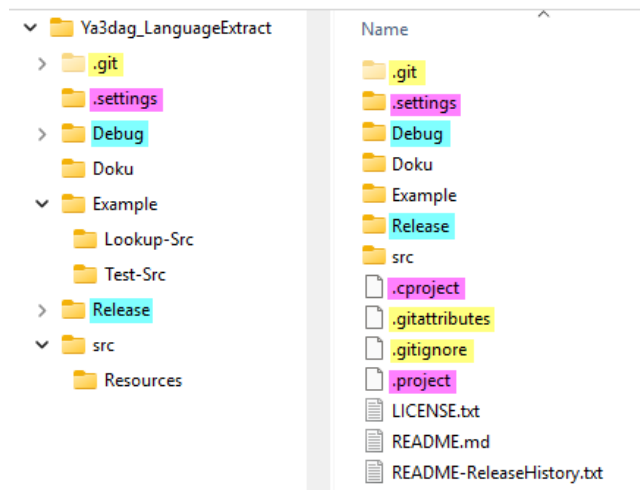
```
pointer to string * If the first character does not start with '&',
                    the 'pString' argument string will be returned.
* If a translation is found, this will be returned.
* If no translation is found, the text following the '='
  character in the 'pString' argument will be returned.
```

NOTE: The translations remain valid as long as no language data is loaded.
This means that after loading language data, the application must be
closed and restarted. For example, after selecting a new language.

```
=====
*/
```

```
char * LangStringLookup( char *pString);
char * LangStringLookup( const char *pString);
```

8. Description of the package's directories



This is a screenshot of my development environment.
The colors have the following meanings:

- Used by Git. Is not important for you.
- Used by Eclipse IDE. Is not important for you.
- Compilation leftovers.

The “**Example**” directory holds the ‘**Ya3dag_LanguageExtract**’ application, a Windows batch file, and several files with language definitions. Double-click the Windows batch file in the file manager!

The “**Lookup-Src**” directory holds the source files for handling language files.

9. Notes on the compiler used

To compile ‘Ya3dag_LanguageExtract’, I use the Eclipse IDE and the “MinGW-W64-builds-5.0.0” compiler.